

基于RISC-V代理内核的操作 系统课程实验与课程设计

第六章 . 实验4: 文件系统

目录

- 实验4的基础知识
 - PKE文件系统架构
 - 文件系统提供的接口
 - 虚拟文件系统VFS
 - RFS文件系统
- 实验内容
 - lab4_1 文件操作
 - lab4_2 目录文件
 - lab4_3 硬链接

实验4的基础知识

PKE lab4的设计目标：为PKE添加文件系统支持

PKE的文件系统应该：

- 1. 为进程提供合适的接口；**
- 2. 支持VFS，以同时实现对host fs文件和定制文件系统的访问；**
- 3. 提供RFS(ramdisk file system)作为实验平台；**

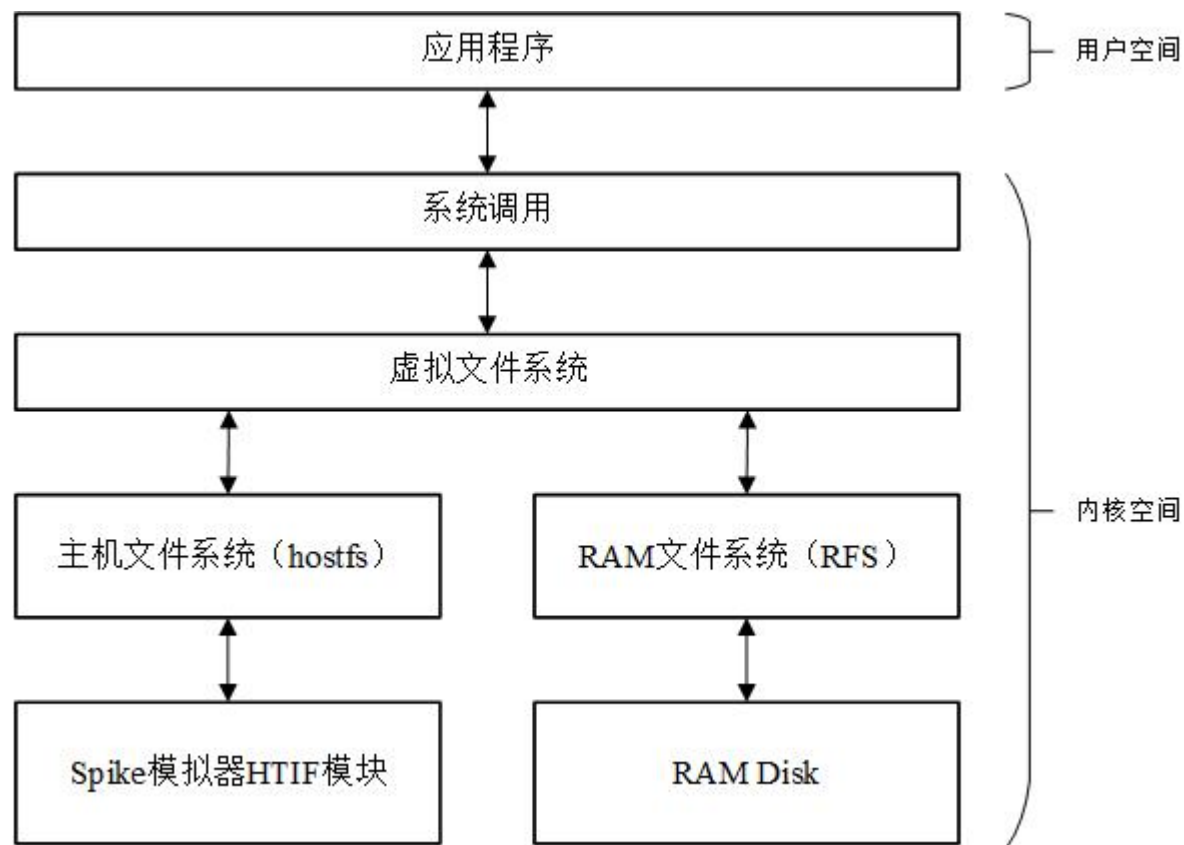
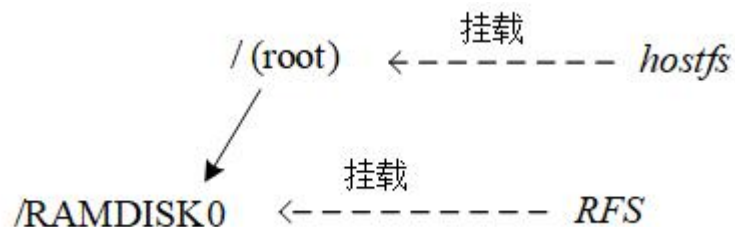
PKE lab4的特点：

**代码量偏大（新增~1700行），因为文件系统是OS最复杂的部分
代码理解阅读需花费较长时间（所以作为课设内容）**

PKE文件系统架构

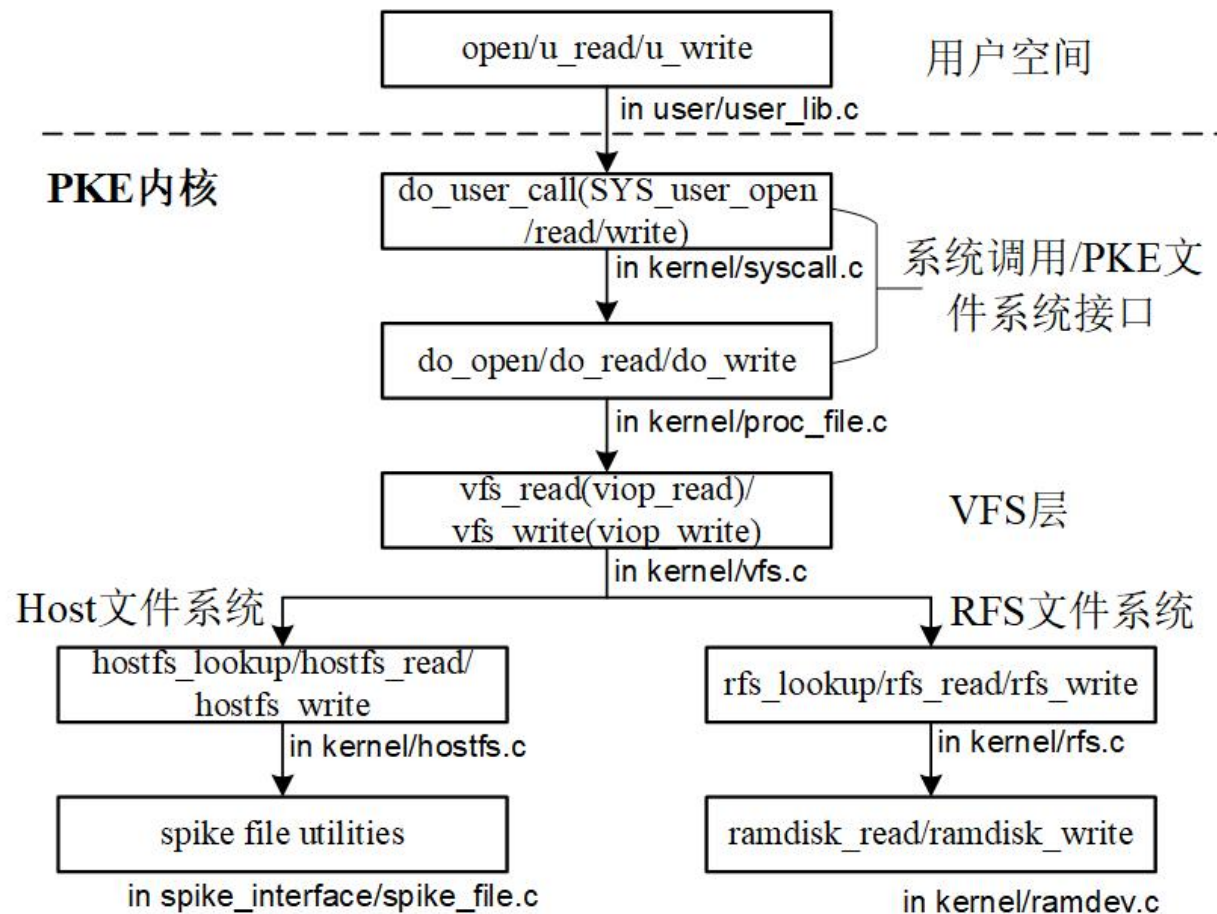
- 特点

- 应用既可以访问hostfs，又可以访问RFS
- 对多个文件系统的访问通过虚拟文件系统进行统一
- 接近真实场景



文件系统的进程接口

- 提供一组用户态函数调用
- OS的系统调用通过VFS层实现
- VFS根据目录位置进行判断,调用具体文件系统的实现完成所需动作



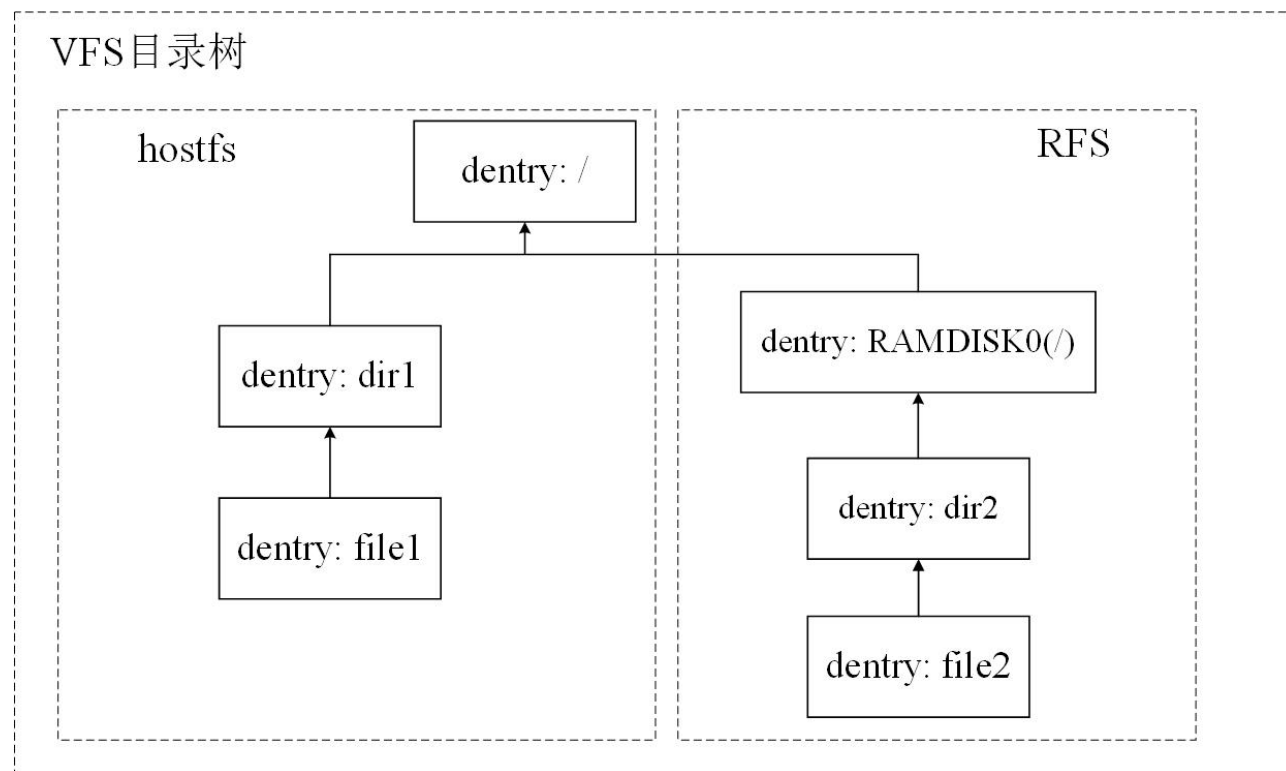
VFS的实现

- VFS的本质是在内存中建立了一套整个文件系统的镜像
- 构造VFS的基础
 - vnode: i节点的抽象
 - dentry: directory entry, 目录在VFS层的抽象
 - super_block: 文件系统相关的信息
 - file: 文件在VFS层的抽象

VFS层的目录组织

- 根目录只有1个
- 跨具体文件系统的存在
- 一切（包括文件）皆dentry
- 采用hash cache做dentry的缓存（提供缓存查找函数）

- /dir1/file1
- /RAMDISK0/dir2/file2



关于RFS（设计思想）

- 分配一段RAM作为Disk（1 disk block=4096 bytes）
- 索引（直接索引）文件系统
- 采用位图法管理Disk空间

RFS的基础构成

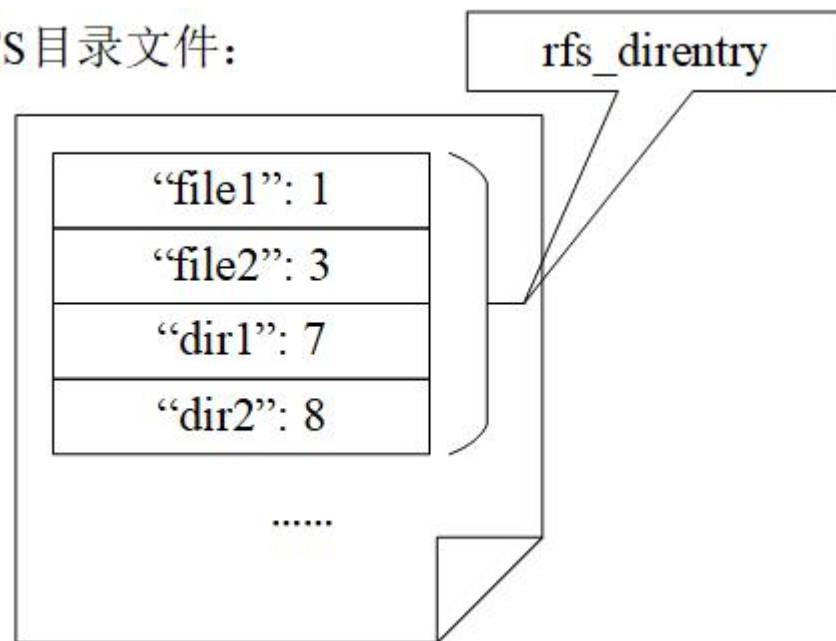
- superblock: 超级块。包含的是文件系统的重要信息，比如disk inode 总个数、块总个数、数据块总个数等等
- dinode: disk inode, “课本上”的“I节点”。包含（直接）索引表
- bitmap: 简单位图

```
/*
 * RFS (Ramdisk File System) is a customized simple file system installed in the
 * RAM disk. added @lab4_1.
 * Layout of the file system:
 *
 * ***** RFS MEM LAYOUT (112 BLOCKS) *****
 *      superblock | disk inodes | bitmap | free blocks *
 *      1 block   | 10 blocks  | 1      | 100          *
 * *****
 *
 * The disk layout of rfs is similar to the fs in xv6.
 */
```

RFS的目录文件

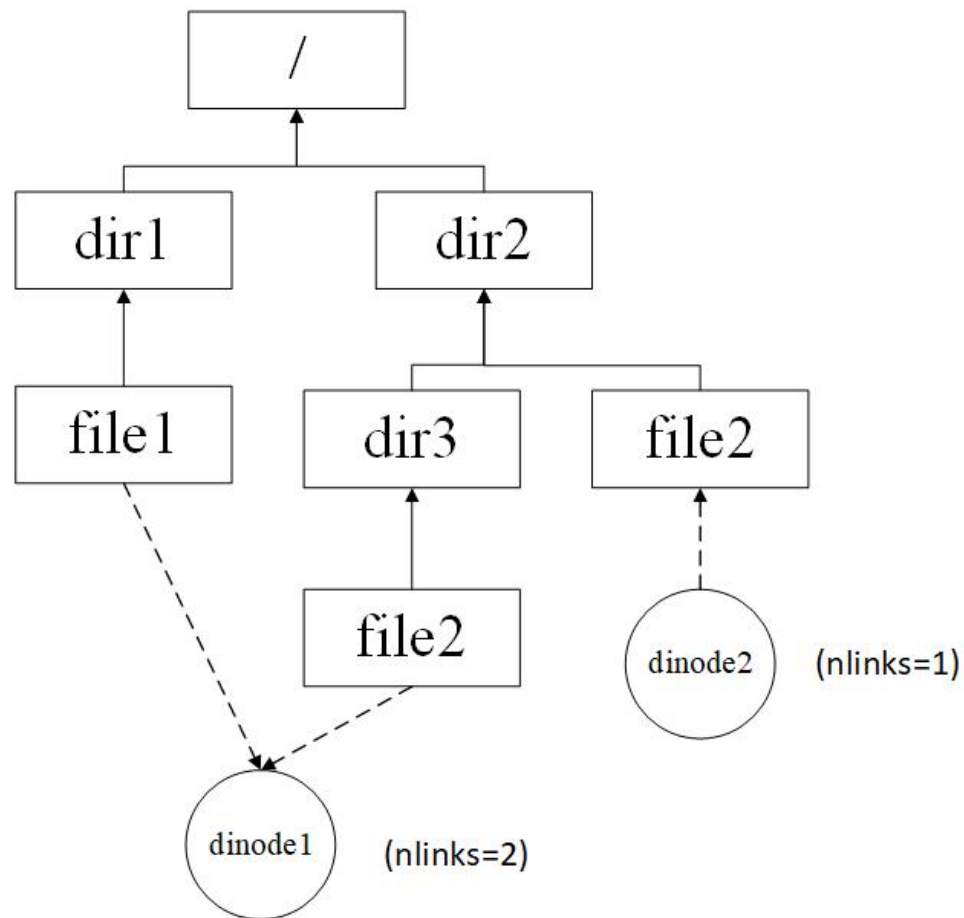
- “课本上”的目录文件
- <名字, dinode编号>的集合

RFS目录文件:



RFS中的硬链接

- “课本上”的硬链接



目录

- 实验4的基础知识
 - PKE文件系统架构
 - 文件系统提供的接口
 - 虚拟文件系统VFS
 - RFS文件系统
- 实验内容
 - lab4_1 文件操作
 - lab4_2 目录文件
 - lab4_3 硬链接

lab4_1 文件操作

• user/app_file.c

```
01 #include "user_lib.h"
02 #include "util/string.h"
03 #include "util/types.h"
04
05 int main(int argc, char *argv[]) {
06     int fd;
07     int MAXBUF = 512;
08     char buf[MAXBUF];
09     char str[] = "hello world";
10     int fd1, fd2;
11
12     printf("\n===== Test 1: read host file =====\n");
13     printf("read: /hostfile.txt\n");
14
15     fd = open("/hostfile.txt", O_RDONLY);
16     printf("file descriptor fd: %d\n", fd);
17
18     read_u(fd, buf, MAXBUF);
19     printf("read content: %s\n", buf);
20
21     close(fd);
22
23     printf("\n===== Test 2: create/write rfs file =====\n");
24     printf("write: /RAMDISK0/ramfile\n");
25
26     fd = open("/RAMDISK0/ramfile", O_RDWR | O_CREAT);
27     printf("file descriptor fd: %d\n", fd);
28
29     write_u(fd, buf, strlen(buf));
30     printf("write content: %s\n", buf);
31     close(fd);
32
33     printf("\n===== Test 3: read rfs file =====\n");
34     printf("read: /RAMDISK0/ramfile\n");
35
36     fd = open("/RAMDISK0/ramfile", O_RDONLY);
37     printf("file descriptor fd: %d\n", fd);
38
39     read_u(fd, buf, MAXBUF);
40     printf("read content: %s\n", buf);
41     close(fd);
42
43     printf("\n===== Test 4: open twice =====\n");
44     printf("file descriptor fd1(ramfile): 0\n");
45     printf("file descriptor fd2(ramfile): 1\n");
46
47     write_u(fd2, str, strlen(str));
48     printf("write content: %s\n", str);
49
50     read_u(fd1, buf, MAXBUF);
51     printf("read content: %s\n", buf);
52     read_u(fd2, buf, MAXBUF);
53     printf("read content: %s\n", buf);
54
55     printf("All tests passed!\n");
56 }
```

给定应用

Application program entry point (virtual address): 0x00000000000100b0
going to insert process 0 to ready queue.
going to schedule process 0 to run.

===== Test 1: read host file =====
read: /hostfile.txt
file descriptor fd: 0
read content:
This is an apple.
Apples are good for our health.

===== Test 2: create/write rfs file =====
write: /RAMDISK0/ramfile
file descriptor fd: 0
write content:
This is an apple.
Apples are good for our health.

===== Test 3: read rfs file =====
read: /RAMDISK0/ramfile
file descriptor fd: 0
read content:
This is an apple.
Apples are good for our health.

===== Test 4: open twice =====
file descriptor fd1(ramfile): 0
file descriptor fd2(ramfile): 1
write content:
hello world
read content:
hello world

All tests passed!

预期输出

lab4_1 文件操作

实验内容：

在PKE操作系统内核中完善对RFS文件的操作，使得它能够正确处理用户进程的打开、创建、读写文件请求。。

修改kernel/rfs.c文件中的rfs_create()函数。填写disk inode完成RFS文件的创建过程。

lab4_2目录文件

- user/app_directory.c

```
01 #include "user_lib.h"
02 #include "util/string.h"
03 #include "util/types.h"
04
05 void ls(char *path) {
06     int dir_fd = opendir_u(path);
07     printu("-----\n");
08     printu("ls \"%s\":\n", path);
09     printu("[name]          [inode_num]\n");
10     struct dir dir;
11     int width = 20;
12     while(readdir_u(dir_fd, &dir) == 0) {
13         // we do not have %ms :(
14         char name[width + 1];
15         memset(name, ' ', width + 1);
16         name[width] = '\0';
17         if (strlen(dir.name) < width) {
18             strcpy(name, dir.name);
19             name[strlen(dir.name)] = ' ';
20             printu("%s %d\n", name, dir.inum);
21         }
22         else
23             printu("%s %d\n", dir.name, dir.inum);
24     }
25     printu("-----\n");
26     closedir_u(dir_fd);
27 }
28
29 int main(int argc, char *argv[]) {
30     char str[] = "hello world";
31     int fd;
32 }
```

给定应用

```
in alloc_proc. build proc_file_management successfully.
User application is loading.
Application: obj/app_directory
CODE_SEGMENT added at mapped info offset:3
DATA_SEGMENT added at mapped info offset:4
Application program entry point (virtual address): 0x0000000000010186
going to insert process 0 to ready queue.
going to schedule process 0 to run.
```

===== Test 1: open and read dir =====

```
-----
ls "/RAMDISK0":
[name]          [inode_num]
-----
```

===== Test 2: make dir =====

```
make: /RAMDISK0/sub_dir
-----
ls "/RAMDISK0":
[name]          [inode_num]
sub_dir         1
-----
```

```
write: /RAMDISK0/sub_dir/ramfile
file descriptor fd: 0
write content:
hello world
-----
```

```
ls "/RAMDISK0/sub_dir":
[name]          [inode_num]
ramfile         2
-----
```

All tests passed!

```
User exit with code:0.
no more ready processes, system shutdown now.
System is shutting down with exit code 0.
```

预期输出

lab4_2目录文件

实验内容：

在PKE操作系统内核中完善对RFS目录文件的访问操作，使得它能够正确处理用户进程的打开、读取目录文件请求。。

完善kernel/rfs.c中的rfs_readdir ()函数，使得该函数能够正确返回dir结构（应用态的ls能够把目录项都列出来）。

lab4_3 硬链接

- user/app_hardlink.c

```
01 #include "user_lib.h"
02 #include "util/string.h"
03 #include "util/types.h"
04
05 void ls(char *path) {
06     int dir_fd = opendir_u(path);
07     printu("-----\n");
08     printu("ls \"%s\":\n", path);
09     printu("[name]          [inode_num]\n");
10     struct dir dir;
11     int width = 20;
12     while(readdir_u(dir_fd, &dir) == 0) {
13         // we do not have %ms :(
14         char name[width + 1];
15         memset(name, ' ', width + 1);
16         name[width] = '\0';
17         if (strlen(dir.name) < width) {
18             strcpy(name, dir.name);
19             name[strlen(dir.name)] = ' ';
20             printu("%s %d\n", name, dir.inum);
21         }
22         else
23             printu("%s %d\n", dir.name, dir.inum);
24     }
25     printu("-----\n");
26     closedir_u(dir_fd);
27 }
28
29 int main(int argc, char *argv[]) {
30     int MAXBUF = 512;
31     char str[] = "hello world";
32     char buf[MAXBUF];
33     int fd = 0;
```

给定应用

```
going to insert process 0 to ready queue.
going to schedule process 0 to run.

===== establish the file =====
create file: /RAMDISK0/ramfile

===== Test 1: hard link =====
create hard link: /RAMDISK0/ramfile2 -> /RAMDISK0/ramfile
file descriptor fd1 (ramfile): 0
file descriptor fd2 (ramfile2): 1
ramfile hard links: 2
/RAMDISK0/ramfile write content:
hello world
/RAMDISK0/ramfile2 read content:
hello world

===== Test 2: unlink =====
-----
ls "/RAMDISK0":
[name]          [inode_num]
ramfile         1
ramfile2        1
-----
unlink: /RAMDISK0/ramfile
-----
ls "/RAMDISK0":
[name]          [inode_num]
ramfile2        1
-----
ramfile2 hard links: 1

All tests passed!

User exit with code:0.
no more ready processes, system shutdown now.
System is shutting down with exit code 0.
```

预期输出

lab4_3 硬链接

实验内容：

在PKE操作系统内核中完善对RFS文件的访问操作，使得它能够正确处理用户进程的创建、删除硬链接的请求。。

完善kernel/rfs.c中的rfs_link()函数，使其能够完成硬链接过程。